Сем3

Сегодня 14:29 Без категории

* Про .bss нужно - я ошибся когда сказал, что зарезервированная память будет раздувать размер бинарного файла - это не так.

Пример type-extension.S -

* Беззнаковое расширение: movzx

- movzx rax, byte [byte_var]

- копирует один байт данных по адресу byte_var остальное дополняется нулями.
- указание размера операнда (спецификатор "byte") при использовании варианта операции память → регистр - обязательно

- movzx rax, bx

- размер обоих операндов известен, его указывать не нужно

- У movzx dest, src:
  dest – всегда регистр
  src – регистр или память с обязательным спецификатором размера

* Знаковое расширение: movsx
  Дополняет недостающие биты копируя знаковый бит src.
  – Все свойства, связанные с операндами аналогичны movzx.

---

* Базовые арифметические операции:
  [Пример math_operations.s]
  - add op1, op2 – сложение

* Что может быть на месте op1 и op2?
  - op1 – регистр, память
  - op2 – константа, регистр, память

! Но, op1 и op2 не могут одновременно быть памятью. Почему?

[Открыть "objdump -d" — дизассемблер

1) Размер инструкции — для старых процессоров это было критическим аргументом

2) По сути операции memory → memory это в любом случае load memory → reg → store memory, но при этом мы потеряли возможность переставить эти обращения в память и спрятали жизнь процессору.

- sub op1, op2 — вычитание
— Ограничения на op1 и op2 аналогичны add

- neg op1 — отрицание
— Унарная операция, т.е. требует только один операнд

• op1 – регистр или память.

Перемещение [пример EFlags.5]

```
mov eax, 0xffffffff
add eax, 1
```

– Что в eax?

– В eax – 0, но, очевидно, что произошло "перемещение".

– А если мы не знаем, после чего eax? Как понять, произошло ли перемещение?

– Ответ: да. Для этого существует регистр EFLAGS.

[Показать картинку, показать в отладчике.

О чем сигнализируют флаги:

• CF – произошло беззнаковое перемещение, т.е. возник перенос из старшего бита

- OF — знаковое переполнение, т.е. итоговой результат не может быть представлен данным числом бит как знаковое число.
- ZF — выставляется, если результат нулевой
- SF — выставляется таким же, как старший бит результата, т.е. по сути символизирует об отрицательности числа

Sleeping-test:

	ZF	SF	OF	CF
1	0	1	1	0
2	1	0	0	1
3	0	0	1	0
4	0	1	0	1

Группа команд для считывания флагов:

setz — ZF

sets — SF
setc — CF
seto — OF

---

* Умножение: mul [mul.s]
mul op

- Умножает eax на op, результат записывает в EDX и EAX: младшая половина в EAX, верхняя половина в EDX
[Показать пример 1]

- В зависимости от размера операнда, mul может вести себя по разному:

Размер op	Неявный второй множитель	Результат
8 бит	AL	AX
16 бит	AX	DX:AX
32 бит	EAX	EDX:EAX

[пример 2, с 8 битным операндом]

• mul может выставлять только флаги CF и OF:
CF=1 и OF=1 — если старшая половина результата ≠0
CF=0 и OF=0 — если старшая половина результата =0

• Для остальных флагов поведение не определено (пример из gdb)

А знаковое умножение: imul
[примеры 1, 2 из imul.S]

Имеет три формы:
I: imul op
II: imul op1, op2 — пример 3
III: imul op1, op2, op3 — пример 4

• Важно: II и III формы не используют EDX, т.е. при переполнении верхняя часть отбрасывается. Вывод =>
=> II и III формой не пользуемся

- Деление: [пример div.s]
  div op

- Делит EDX:EAX на op.
  В EAX пишет частное, в EDX - остаток.
- Размеры операндов такие, как у mul.

[Пример 1, показать деление на 0

* Знаковое деление: idiv
- Т.к. требуется знаковое число в EDX:EAX, обязательно знаковое расширение:
  cbw 8 → 16
  cwd 16 → 32
  cdq 32 → 64

* Важно: div / idiv не выставляют никаких флагов в EFLAGS

Дополнение про флаг OF:

- OF выставляется, если результат нашей операции не пролезет диапазону $[-INT_MIN, INT_MAX]$

- Пример 1 из eflags.s

```
mov eax, 0x7fffffff
add eax, 1
```

положительное знаковое число

$0x7fffffff = 2^{31} - 1$

$\Rightarrow 2^{31} - 1 + 1 = 2^{31} \notin [-2^{31}; 2^{31} - 1]$

$\Rightarrow OF = 1$

- Пример 2:

```
mov eax, 0xffffffff
add eax, 1
```

отрицательное число

$\Rightarrow 0xffffffff = -1$

$\Rightarrow (-1) + 1 = 0 \in [-2^{31}; 2^{31} - 1]$

$\Rightarrow OF = 0$

- Пример 3:

```
mov eax, 0x80000000
```

mov eax, 0x80000000

sub eax, 1

↑ отриц. знаковое число

$0x80000000 = -2^{31}$

$\Rightarrow -2^{31} - 1 \notin [-2^{31}; 2^{31} - 1]$

$\Rightarrow OF = 1$

- Пример 4:

mov eax, 0

sub eax, 1

$\Rightarrow 0 - 1 = -1 \in [-2^{31}; 2^{31}] \Rightarrow$

$\Rightarrow OF = 0$